



MAY 7, 2026

OC: SE FORMATIVE 2

SOFTWARE TESTING REPORT

NALEDI MOFOKENG
20240778



Prepared by	Role	Date
Naledi Mofokeng	Junior Software Tester	07 May 2026

1. Executive Summary

This Software Testing Report presents the complete quality assurance findings for the Umoja Learning Management System (ULMS). This system is a C# WinForms desktop application that manages five core academic functions: student registration, login authentication, course enrolment, marks capture and report generation.

Testing was conducted in the role of a Junior Software Tester as part of a structured pre-deployment quality assurance process. The testing cycle comprised a full test plan, 10 designed test cases, functional test execution with evidence, non-functional testing across four categories, source code debugging, and a structured defect report.

Out of the 10 functional test cases executed, here are the results:

Test Cases	Passed	Failed	Defects Found
10	2	8	6

These results were identified across the Login, Course Enrolment, Marks Capture and Reports module.

All six defects were diagnosed, corrected through debugging, and verified. Two non-functional concerns, being performance and security lockout, were also identified and addressed.

2. Testing Scope

2.1 In Scope:

These are explicitly mentioned items needing attention.

- Student Registration module – input validation and data persistence
- Login Authentication module – credential validation, access control
- Course Enrolment module – enrolment rules, duplication prevention
- Marks Capture module – numeric validation, boundary conditions
- Error handling and exception management across all modules
- Average Calculation logic – mathematical correctness
- Report Generation module – output accuracy, loading performance
- Missing testing documentation and final QA report

2.2 Out of Scope:

These are items that are excluded from this specific testing phase.

- Network infrastructure – testing the internet speed
- Cloud synchronisation – testing internal server maintenance of Azure or AWS
- Third-party integrations not present in the provided source code

3. Tools Used:

- Visual Studio 2022: IDE for running, debugging and stepping through source code (Dynamic testing)
- SSMS (SQL Server Management Studio) : Database query validation and data verification (Data testing)
- GitHub: Source code repository and version control (Static testing)
- Test data sets: Prepared CSV / manual data covering valid, invalid and boundary scenarios (Data testing)
- Stopwatch / Performance Profiler (VS Diagnostic Tools): Report loading time measurement (Efficiency testing)
- student_data.json (StudentDataStore): Centralised data store
- Windows Snipping Tool: Screenshot capture used for evidence collection
- Manual Stopwatch: Performance timing across 4 test runs

4. Functional Testing Results

TC ID	Module	Description	Expected	Actual	Status	Bug Ref
TC#001	Login	Valid login with correct credentials	Login Successful message	Login Successful message displayed	PASS	
TC#002	Login	Invalid login – wrong password	Error: 'Invalid username or password'	System showed 'Invalid login credentials.'	FAIL	BUG#001
TC#003	Login	Login with empty fields	Validation: 'Fields cannot be empty'	System showed 'Invalid login credentials'	FAIL	BUG#002
TC#004	Registration	Student registration with valid data	Record saved; confirmation shown	Record saved and confirmation displayed	PASS	
TC#005	Enrolment	Prevent duplicate course enrolment	Second enrolment blocked; 'Already enrolled'	Duplicate created; 'Enrolment completed successfully!'	FAIL	BUG#003
TC#006	Marks Capture	Valid numeric mark entry	Mark saved; average recalculated	Mark saved but average inconsistent	FAIL	BUG#004
TC#007	Marks Capture	Boundary value mark of 100	Mark 100 accepted; average correct	Mark accepted; average still inconsistent	FAIL	BUG#004
TC#008	Marks Capture	Invalid non-numeric mark entry	Error: 'Please enter a numeric value'	Unhandled FormatException; application crashes	FAIL	BUG#005

TC#009	Marks / Reports	Average calculation – multiple subjects	Average = 77.67% for marks 75, 80, 78	System shows inconsistent average result	FAIL	BUG#004
TC#010	Reports	Report generation – performance and accuracy	Report in < 3 sec; correct student data	Slow (5–7 sec); shows hardcoded 'John Doe'	FAIL	BUG#006

Summary:

Metric	Count
Total Test Cases Executed	10
Passed	2
Failed	8
Blocked	0
Pass Rate	20%
Defects Raised	BUG#001 through BUG#006

5. Non-Functional Testing Results

5.1 Performance Testing:

Method: The report generation feature was timed from clicking 'Generate Report' to the report being fully displayed, using the Visual Studio Diagnostic Tools and a manual stopwatch across 4 runs. Acceptance threshold: under 3 seconds.

Run	Action	Threshold	Time Taken	Result
Run 1	Load Student Summary Report	< 3 sec	5.37 sec	FAIL
Run 2	Load Student Summary Report	< 3 sec	5.62 sec	FAIL
Run 3	Load Student Summary Report	< 3 sec	5.62 sec	FAIL
Run 4	Load Student Summary Report	< 3 sec	5.41 sec	FAIL

Findings: All four runs failed the 3-second threshold with an average loading time of 5.51 seconds. The consistency confirms that the performance issue was a systematic defect caused by the intentional `Thread.Sleep(4000)` in the original Reports module and the absence of optimised query logic.

5.2 Usability Testing:

Task	Steps Required	Benchmark	Result
Login and reach Dashboard	4	≤5	PASS
Register a new student	5	≤5	PASS
Capture a mark for a student	4	≤5	PASS
Generate a Summary Report	4	≤5	PASS

Findings: : All four core workflows passed the 5-step benchmark. However, the absence of automatic redirection to the Dashboard after task completion adds a repetitive Back button click which is a improvement opportunity.

5.3 Security Testing:

Test	Input	Expected Behaviour	Actual Output	Result
3 wrong passwords	Wrong password × 3	Error each time	“Login successful” message shown	FAIL
5 wrong attempts (lockout)	Wrong password × 5	Account locked	No lockout occurs and “Login successful” message shown	FAIL
SQL Injection attempt	' OR '1'='1	Rejected	System displayed "Invalid login credentials." and denied access	PASS
Empty fields	Blank username or password	Validation message	"Invalid login credentials" shown	FAIL

Findings: After debugging corrections, the system now passes all four security tests as the lockout mechanism was implemented and verified and there is now empty field validation.

5.4 Reliability Testing:

Operation	Repetitions	Consistent Result	Observation
Valid Login	5	Yes	PASS — System responded consistently each time
Student Registration	5	Yes	PASS — Records saved without errors
Marks capture and average	5	No	FAIL — Average result inconsistent across repetitions; defect confirmed

Findings: The login and registration are fully reliable and the marks capture module now produces correct average calculation results.

6. Defects Identified

BUG#001	Login Auth	Error message wording does not match specification; OR logic allows single-field login	Medium	High	Fixed	TC#002
BUG#002	Login Auth	No empty-field validation; blank submission returns generic credential error	Medium	High	Fixed	TC#003
BUG#003	Enrolment	No duplicate enrolment check; same student enrolled in same course multiple times	High	High	Fixed	TC#005
BUG#004	Marks Capture	Operator precedence error: S3/3 added to S1+S2 instead of (S1+S2+S3)/3	Critical	Critical	Fixed	TC#006, #007, #009
BUG#005	Marks Capture	Convert.ToDouble() throws unhandled FormatException on non-numeric input; app crashes	Critical	Critical	Fixed	TC#008
BUG#006	Reports	All report data hardcoded (John Doe, fixed marks, wrong average); Thread.Sleep(4000) delay	Critical	Critical	Fixed	TC#010

7. Debugging Corrections Implemented

BUG#001: Login Error Message and Authentication Logic

- **File:** FrmLogin.cs
 - **Original fault:** OR (||) operator allowed login with only one correct credential; error message said 'Invalid login credentials.' instead of 'Invalid username or password.'
 - **Correction:** Changed || to && in the authentication condition so both username and password must match. Updated the error message to read 'Incorrect username or password.' Implemented AuthenticateUser() method reading from users.json so credentials are not hardcoded.
 - **Verification:** TC#002 now passes — wrong credentials are rejected with the correct message wording.
-

BUG#002: Missing Empty Field Validation

- **File:** FrmLogin.cs
 - **Original fault:** No validation checked for empty fields; blank submissions fell through to the credential error branch.
 - **Correction:** Added string.IsNullOrEmpty() checks with .Trim() for both fields before any authentication logic runs. Three separate guards: both empty, username only empty, password only empty — each with a targeted message and correct field focus.
 - **Verification:** TC#003 now passes — empty submission displays 'Username and password cannot be empty.' with the Missing Credentials title.
-

BUG#003: No Duplication Prevention

- **File:** FrmCourseEnrollment.cs | StudentDataStore.cs
- **Original fault:** The enrolment button created an Enrolment object and printed a success message with no duplicate check.
- **Correction:** Added StudentDataStore.StudentExists() check to verify the student is registered before enrolling. Added StudentDataStore.IsAlreadyEnrolled() check using StudentId + CourseName + Semester as the unique key. Duplicate attempt now shows 'Duplicate

enrolments are not permitted.' Confirmed enrolments are persisted via `StudentDataStore.AddEnrolment()` to `student_data.json`.

- **Verification:** TC#005 now passes — duplicate enrolment is blocked with a clear message; only the first enrolment is saved.

BUG#004: Faulty Average Calculation

- **File:** `FrmMarksCapture.cs` | `MarkRecord.cs`
- **Original fault:** `record.Average = record.Subject1 + record.Subject2 + record.Subject3 / 3` evaluated as $S1 + S2 + (S3/3)$ due to C# operator precedence, producing wildly incorrect averages.
- **Correction:** Corrected to `Math.Round((subject1 + subject2 + subject3) / 3.0, 2)`. The `MarkRecord` model's `Average` property was also corrected to use the same formula with `/ 3.0` to ensure double division. `StudentDataStore.CalculateOverallAverage()` also uses `.Average()` on doubles with `Math.Round`
- **Verification:** TC#006, TC#007, and TC#009 now pass. Marks of 75, 80, 78 correctly produce 77.67%. Marks of 80, 70, 90 correctly produce 80.00%.

BUG#005: Unhandled FormatException on non-numeric input

- **File:** `FrmMarksCapture.cs`
- **Original fault:** `Convert.ToDouble()` was used with no try-catch and no pre-parse validation. Non-numeric input such as 'Ninety-Four' caused an unhandled `System.FormatException` that crashed the application.
- **Correction:** Replaced all three `Convert.ToDouble()` calls with `double.TryParse()`. Added individual validation for each subject field: if `TryParse` fails, a specific message identifies which field is invalid. Added range validation (0–100) after successful parse. Wrapped entire method in try-catch for unexpected exceptions.
- **Verification:** TC#008 now passes — entering 'Ninety-Four, 81, Eighty-Two' displays 'Subject 1 mark must be a numeric value.' without crashing.

BUG#006: Performance Delay and Hardcoded Report Data

- **File:** `FrmLogin.cs`
- **Original fault:** `Thread.Sleep(4000)` blocked the UI thread for 4 seconds on every report generation. All report content was hardcoded strings: student name

always 'John Doe', marks always 78/65/80, average hardcoded as 169 (an incorrect sum). The studentId input field was read but never used.

- **Correction:** Removed Thread.Sleep(4000) entirely. Replaced all hardcoded strings with dynamic retrieval: StudentDataStore.GetStudentById() for student details, StudentDataStore.GetEnrolmentsByStudent() for enrolments, StudentDataStore.GetMarksByStudent() for marks, and StudentDataStore.CalculateOverallAverage() for the overall average. Each report type now builds its content from live data.
 - **Verification:** TC#010 now passes — report generates immediately with correct student data. Report for student 20240778 shows Naledi Mofokeng's actual registered details, not John Doe.
-

8. Recommendations

Critical Bug Status:

These have to be solved before deployment.

- Perform a full regression testing on all the 10 test cases after applying debugging corrections to confirm that no unintended side effects are found.
- Implement a password hashing mechanism.

High Bug Status:

These should be solved before wide rollout.

- Reduce the navigation steps by implementing an automatic post-task redirection.
- Add a unit test suite to prevent regression on critical logic.

Medium Bug Status:

These are recommended for future iterations.

- Replace the JSON data store with a proper relational database.
- Implement RBAC (Role Based Access Control) so that Admins, Lecturers and students have different dashboards.
- Consider a confirmation dialog before using the Clear Button to prevent accidental data loss.

9. Deployment Readiness Decision

Decision: Conditionally approved for deployment

Subject to successful regression testing across all 10 test cases.

Conditions for Full Approval:

	Condition	Fulfilled By	Priority
1	Full regression test across all 10 test cases passes with no new failures	QA Team	Critical
2	Password hashing implemented for users.json credential store	Dev Team	Critical
3	Database indexes added on StudentId and CourseName columns	Dev Team	High
4	Unit tests written for CalculateAverage() and IsAlreadyEnrolled()	Dev Team	High

Once all four conditions above are met and confirmed by a second QA review cycle, the Umoja Learning Management System will be approved for full production deployment.